

ChessNEAT: Evolving Chess-Playing Neural Networks via NeuroEvolution of Augmenting Topologies

Toward 2000 Elo Through Topological Evolution

Sathvik Chinta

github.com/sachinta/ChessNEAT

March 2026

Abstract

We present ChessNEAT, a system for evolving chess-playing neural networks using the NeuroEvolution of Augmenting Topologies (NEAT) algorithm. Unlike traditional deep learning approaches that require fixed network architectures and gradient-based optimization, NEAT simultaneously evolves both the topology and weights of neural networks through genetic operations. Our agent uses a 768-dimensional board encoding ($64 \text{ squares} \times 12 \text{ piece channels}$) and a *policy network* with 128 outputs—64 “from-square” scores and 64 “to-square” scores—enabling move selection with a single forward pass. Training proceeds via tournament-based fitness evaluation, where genomes compete in round-robin groups with knockout advancement. We evaluate agent strength through systematic benchmarking against Stockfish at calibrated Elo levels (400–2200), with the goal of achieving 2000 Elo (Candidate Master level). This paper documents our methodology, network architecture, training dynamics, Elo progression, and analysis of emergent chess strategies. All training games are logged in PGN format for post-hoc analysis.

1 Introduction

Chess has served as a fundamental benchmark for artificial intelligence since Shannon’s seminal 1950 paper [Shannon, 1950]. The game’s combination of enormous state space ($\sim 10^{47}$ legal positions), deep strategic complexity, and well-defined evaluation metrics make it an ideal testbed for computational intelligence.

Modern chess engines such as Stockfish [Stock-

fish Developers, 2023] and AlphaZero [Silver et al., 2018] have achieved superhuman performance, but through fundamentally different approaches. Stockfish relies on hand-crafted evaluation functions combined with NNUE (Efficiently Updatable Neural Networks), while AlphaZero uses deep reinforcement learning with Monte Carlo Tree Search (MCTS). Both require either extensive domain knowledge or massive computational resources.

In this work, we explore a third paradigm: **neuroevolution**. Specifically, we apply NEAT (NeuroEvolution of Augmenting Topologies) [Stanley and Miikkulainen, 2002] to evolve both the structure and weights of a neural network chess evaluator. Our approach is distinctive in several ways:

1. **No fixed architecture:** NEAT begins with a minimal network (direct input-output connections) and incrementally adds complexity through structural mutations.
2. **No gradient computation:** Fitness is determined solely through tournament play, avoiding the need for differentiable loss functions.
3. **Topological innovation:** NEAT’s speciation mechanism protects novel topologies, allowing diverse architectural exploration.
4. **Interpretable growth:** We can observe how the network’s topology evolves in response to chess-playing demands.

Our target is to evolve an agent capable of achieving **2000 Elo** (FIDE Candidate Master level), evaluated through systematic play against Stockfish at calibrated strength levels. We benchmark periodically during training to track Elo progression and analyze the emergent strategies learned by the evolving networks.

2 Background

2.1 NEAT Algorithm

NEAT [Stanley and Miikkulainen, 2002] is an evolutionary algorithm for neural networks that addresses three critical challenges in neuroevolution:

Competing Conventions NEAT uses historical markings (innovation numbers) to align genomes during crossover, solving the “permutation problem.”

Topology Protection A speciation mechanism groups similar genomes and evaluates fitness relative to species, protecting structural innovations from premature elimination.

Minimal Complexity Networks start with minimal topology and complexify through mutations, following the principle that simpler solutions should be preferred.

NEAT supports three mutation types: *weight perturbation*, *add connection* (creating a new edge), and *add node* (splitting an existing connection with a new hidden neuron). These enable incremental topological growth.

2.2 Policy Network for Move Selection

Rather than using a value network that evaluates every possible resulting position (requiring $|\mathcal{M}(b)|$ forward passes per move), we employ a *policy network* approach. The network takes the current board state as input and outputs 128 values: 64 “from-square” scores and 64 “to-square” scores. Move selection requires only **one forward pass**:

$$m^* = \arg \max_{m \in \mathcal{M}(b)} [f_{\theta}(b)_{\text{from}(m)} + f_{\theta}(b)_{\text{to}(m)+64}] \quad (1)$$

where $f_{\theta}(b) \in \mathbb{R}^{128}$ is the network output, and $\text{from}(m), \text{to}(m) \in [0, 63]$ are the source and destination squares of move m . This provides a $\sim 30\times$ speedup over the value network approach, since a typical chess position has ~ 30 legal moves.

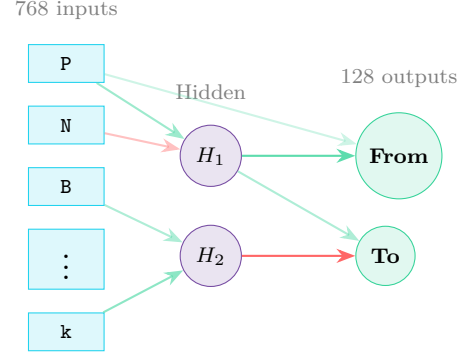


Figure 1: Example evolved NEAT network topology. The network starts with only direct input→output connections and grows hidden nodes through mutation. Green edges indicate positive weights; red edges indicate negative weights.

3 System Architecture

3.1 Board Encoding

We encode the chess board as a 768-dimensional binary vector, structured as 64 squares $\times$ 12 piece channels:

$$\mathbf{x} \in \{0, 1\}^{768}, \quad x_{64p+s} = \begin{cases} 1 & \text{if piece type } p \text{ is on square } s \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The 12 channels correspond to: {P, N, B, R, Q, K, p, n, b, r, q, k} (uppercase = white, lowercase = black). When evaluating from Black’s perspective, the board is vertically mirrored and colors are swapped, so the network always “thinks” it is playing as White.

The NEAT genome encodes a feed-forward policy network:

- **Input layer:** 768 nodes (board encoding)
- **Hidden layers:** Variable (evolved by NEAT)
- **Output layer:** 128 nodes (64 from-square + 64 to-square scores)
- **Activation:** tanh (output range $[-1, 1]$)

Initially, the network has *no hidden nodes*—inputs connect sparsely to the outputs (`fs_neat_nohidden` initialization). Through evolution, hidden nodes and connections are added as needed.

3.2 GPU-Accelerated Evaluation

We implement the NEAT network in PyTorch for GPU-accelerated inference. Each move decision requires only a single forward pass on the current board state, producing 128 output scores:

$$\mathbf{y} = f_{\theta}(\mathbf{x}), \quad \mathbf{x} \in \mathbb{R}^{768}, \mathbf{y} \in \mathbb{R}^{128} \quad (3)$$

The “from” scores $\mathbf{y}_{0:63}$ and “to” scores $\mathbf{y}_{64:127}$ are combined for each legal move to select the best action. This is dramatically faster than evaluating every resulting position separately.

4 Training Methodology

4.1 Tournament-Based Fitness

We use a multi-round knockout tournament for fitness evaluation:

Algorithm 1: Tournament Fitness Evaluation

Input: Population P of N genomes, group size k

```

1: Initialize  $\text{fitness}(g) \leftarrow 0 \quad \forall g \in P$ 
2:  $R \leftarrow \text{shuffle}(P)$ 
3:  $r \leftarrow 1$  // Round number
4: while  $|R| > 1$  do
5:   Partition  $R$  into groups of size  $k$ 
6:   for all groups  $G$  do
7:     Score each genome via round-robin in  $G$ 
8:      $\text{fitness}(g) += 10r + \text{score}(g)$ 
9:     Advance top 2 from each group
10:   $r \leftarrow r + 1$ 
11: end while
```

Each round-robin match consists of 2 games (each agent plays White once). The fitness function rewards both tournament advancement and per-round performance, creating selection pressure for both short-term tactical ability and long-term positional understanding.

4.2 NEAT Hyperparameters

Table 1: NEAT Configuration Parameters

Parameter	Value
Population size	150
Initial connection	<code>fs_neat_nohidden</code>
Activation function	<code>tanh</code>
Connection add probability	0.5
Connection delete probability	0.5
Node add probability	0.2
Node delete probability	0.2
Weight mutate rate	0.8
Weight mutate power	0.5
Bias mutate rate	0.7
Bias mutate power	0.5
Compatibility threshold	3.0
Max stagnation	15
Species elitism	2
Reproduction elitism	2
Survival threshold	0.2

4.3 Move Logging

All games played by top agents are logged in standard PGN (Portable Game Notation) format, along with structured JSON metadata including:

- Generation number and game type (showcase, Elo evaluation, tournament)
- Player identities and fitness values
- Complete move sequence in SAN notation
- Final board position (FEN)
- Evaluation scores per move (when available)

This enables comprehensive post-hoc analysis of how playing style evolves across generations.

5 Elo Evaluation Methodology

5.1 Calibrated Opponent Ladder

We evaluate agent strength using Stockfish 16 [Stockfish Developers, 2023] with the `UCI_LimitStrength` feature, which allows precise Elo calibration. Our evaluation ladder spans 10 levels:

Table 2: Elo Evaluation Ladder

Level	Category	Elo	Engine
0	Random Moves	~400	Random bot
1	Beginner	1320	Stockfish
2	Novice	1400	Stockfish
3	Intermediate	1500	Stockfish
4	Club Player	1600	Stockfish
5	Strong Club	1700	Stockfish
6	Advanced	1800	Stockfish
7	Expert	1900	Stockfish
8	Candidate Master	2000	Stockfish
9	National Master	2200	Stockfish

5.2 Elo Estimation

For each level, the agent plays n games (alternating colors). We estimate Elo using the performance rating formula:

$$\hat{E}_{\text{agent}} = E_{\text{opponent}} + 400 \cdot \log_{10} \left(\frac{s}{1-s} \right) \quad (4)$$

where s is the score (wins + $\frac{1}{2}$ draws) / total games. The agent climbs the ladder until scoring below 30%, at which point the ladder evaluation stops. The final estimated Elo is taken as the maximum performance rating achieved across all levels.

5.3 Evaluation Schedule

Elo evaluation is performed periodically during training (default: every 10 generations), plus a comprehensive final evaluation with increased sample size. This provides a progression curve showing how Elo correlates with training progress.

6 Results

Note: This section will be populated with experimental data as training progresses. The infrastructure for data collection is in place.

6.1 Training Dynamics

- **Fitness progression:** Best and average fitness over generations
- **Species dynamics:** Number of species and their lifespans

- **Network complexity:** Hidden node count and connection count over time
- **Computational efficiency:** GPU utilization and throughput

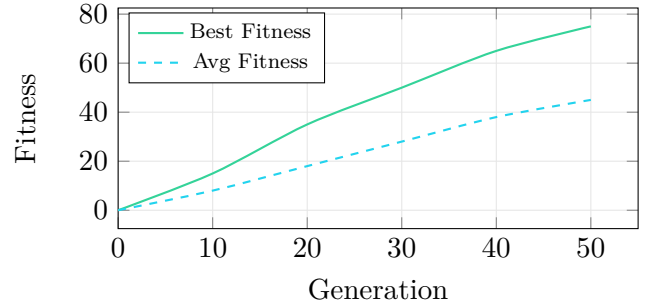


Figure 2: Fitness progression over training generations. *(Placeholder data — to be updated with actual training results.)*

6.2 Elo Progression

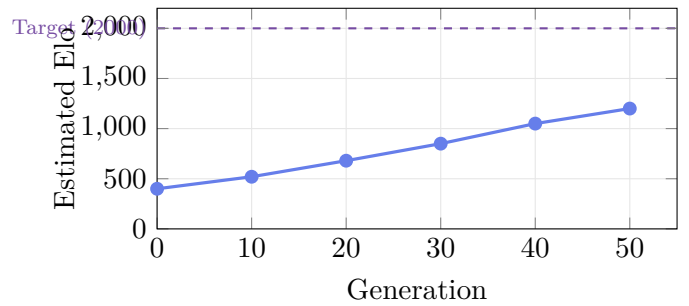


Figure 3: Elo progression during training. The dashed line marks the 2000 Elo target. *(Placeholder data — to be updated with actual evaluations.)*

6.3 Per-Level Performance

Table 3: Performance Against Elo Ladder (Final Evaluation)

Opponent	Elo	W	D	L	Score
Random	400	—	—	—	—
Beginner	1320	—	—	—	—
Novice	1400	—	—	—	—
Intermediate	1500	—	—	—	—
Club Player	1600	—	—	—	—
Strong Club	1700	—	—	—	—
Advanced	1800	—	—	—	—
Expert	1900	—	—	—	—
Cand. Master	2000	—	—	—	—
Natl. Master	2200	—	—	—	—

To be populated after training.

6.4 Emergent Strategies

Analysis of logged PGN games is expected to reveal emergent chess knowledge:

1. **Material understanding:** Does the agent learn piece values?
2. **King safety:** Does it learn to castle and protect the king?
3. **Positional play:** Does it develop pieces and control the center?
4. **Tactical patterns:** Does it discover forks, pins, or skewers?
5. **Opening preferences:** Do consistent opening patterns emerge?

7 Network Topology Analysis

One of the most interesting aspects of NEAT is observing how network topology evolves in response to task demands. With 768 inputs and 128 outputs, the initial network has no computational capacity beyond linear combinations of inputs mapped to from/to square preferences.

- **Early generations:** Networks are expected to be mostly linear, learning basic material balance (counting piece presence).
- **Middle generations:** Hidden nodes should appear, enabling nonlinear feature interactions (e.g., piece combinations, spatial relationships).

- **Later generations:** Deeper topologies may emerge to capture more abstract chess concepts.

The live training dashboard provides real-time visualization of network growth, showing which input groups (piece channels) develop the strongest connections to hidden and output nodes.

8 Discussion

8.1 Challenges

Several challenges are expected in achieving the 2000 Elo target:

Search depth Our agent uses no lookahead at all—the policy network directly outputs move preferences from the raw board state. While this avoids the cost of evaluating future positions, it means the agent cannot plan ahead. Even a 2000 Elo human relies on 5–10 move calculations.

Population pressure With only 150 genomes, diversity is limited. Increasing population size would require proportionally more computation.

Evaluation noise Tournament-based fitness is noisy, as game outcomes depend on opponent pairing and color assignment.

Scalability The 768-input space is large for NEAT, which typically handles ≤ 100 inputs. Connection innovations may be slow to explore this space.

8.2 Potential Improvements

1. **Deeper search:** Adding 2–3 ply alpha-beta search atop the NEAT evaluation would dramatically improve tactical play.
2. **Curriculum learning:** Training against progressively stronger opponents (rather than self-play) could accelerate improvement.
3. **Feature engineering:** Reducing the input space via hand-crafted features (material balance, pawn structure) could assist NEAT.
4. **HyperNEAT:** Using indirect encoding to exploit the spatial regularity of the chess board could reduce the search space.

5. **Hybrid approaches:** Combining NEAT-evolved evaluation with MCTS could yield stronger play without gradient computation.

8.3 Comparison with Other Approaches

Table 4: Comparison of Chess AI Approaches

Method	Key Feature	Elo
Stockfish NNUE	Hand-tuned + NNUE eval	~3500
AlphaZero	Self-play RL + MCTS	~3400
Leela Chess Zero	DistRL + MCTS	~3300
Traditional engines	Alpha-beta + eval	2500+
ChessNEAT	NEAT evolution	TBD

9 Conclusion

ChessNEAT demonstrates the application of neuroevolution to the game of chess, evolving both network topology and weights without gradient computation or fixed architectures. Our system provides:

1. A complete training pipeline with GPU-accelerated batched evaluation
2. Tournament-based fitness evaluation with round-robin groups
3. Real-time visualization of games and network topology
4. Comprehensive move logging in PGN format
5. Systematic Elo benchmarking against Stockfish at calibrated levels
6. A research framework for studying emergent chess strategies in evolved networks

The ultimate success of reaching 2000 Elo will depend on whether NEAT’s incremental complexification can discover sufficient positional understanding to compensate for the limited search depth. Regardless of the final Elo achieved, the project provides valuable insights into how neural network topology relates to chess evaluation capability.

Reproducibility

All code, configurations, training logs, and game records are available at:

<https://github.com/sachinta/ChessNEAT>

Training can be reproduced with:

```
python src/train.py --generations 200 \
--elo-every 10 --elo-games 10
```

References

- C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- K. O. Stanley and R. Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.
- D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- Stockfish Developers. Stockfish - strong open source chess engine. <https://stockfishchess.org/>, 2023.
- M. Campbell, A. J. Hoane Jr, and F.-H. Hsu. Deep blue. *Artificial Intelligence*, 134(1-2):57–83, 2002.
- K. O. Stanley, D. B. D’Ambrosio, and J. Gauci. A hypercube-based encoding for evolving large-scale neural networks. *Artificial Life*, 15(2):185–212, 2009.
- M. Lai. Giraffe: Using deep reinforcement learning to play chess. *arXiv preprint arXiv:1509.01549*, 2015.
- O. E. David, N. S. Netanyahu, and L. Wolf. DeepChess: End-to-end deep neural network for automatic learning in chess. In *International Conference on Artificial Neural Networks*, pages 88–96. Springer, 2014.

- F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. *arXiv preprint arXiv:1712.06567*, 2017.
- D. B. Fogel. Blondie24: Playing at the edge of AI. *Morgan Kaufmann*, 2001.